

게이팅 단일화 — 완료 보고

세 곳의 게이트와 두 벌의 쿨 상태를 AbilityRunner 한 권위로. 쿨=절대시각, 코스트만 위임, UI는 러너 폴링. 죽은 코드 정리까지.

한 줄 요약 — CharacterCombat 평타레이트 · CharacterStat Q/E 쿨 · AbilityRunner 쿨로 흩어져 있고 두 벌로 평행하던 쿨 상태를 AbilityRunner 한 권위로 모았다. 쿨은 절대시각(IClock)으로만 존재 → PartyManager.TickCooldowns 삭제. 코스트(궁게이지)만 Stat에 남기고 러너가 위임 소비, UI(쿨 채움·궁 준비)는 러너를 폴링.

계획: Phase1_Plan.pdf · 선행: Phase0_Result.md

01 AbilityRunner — admission / fire 분리 + 읽기 API

TryCast 한 덩이를 셋으로. 몸/보스 호출부는 TryCast (=Commit+Fire)라 무변경.

```
// 게이트+커밋: 통과 시 쿨만 시작(이펙트 X). 입력 시점 상태 전이 결정용
public bool Commit(AbilityData a, AbilityContext ctx);
// 비게이트: 캐스트타임 + 이펙트 + GrantsTags try/finally
public UniTask Fire(AbilityData a, AbilityContext ctx, CancellationToken token);
// 편의(몸/보스/E 즉발): Commit 성공 시 Fire
public UniTask TryCast(AbilityData a, AbilityContext ctx, CancellationToken token);
// 읽기 API - 쿨 상태 _cd:(end,dur) 저장 → CDR에도 채움비율 정확
public float CooldownRemaining(AbilityData a);
public float CooldownDuration(AbilityData a);
```

02 쿨 값의 출처 = CharacterDataSO (러너는 모름)

쿨 "값"은 SO에 두고 캐스터가 ctx.CoolDown 으로 건넨다. 러너는 결합되지 않는다.

```
// AbilityRunner.StartCooldown
float cd = ctx.CoolDown ≥ 0f ? ctx.CoolDown : a.CoolDown; // 몸은 안 건넨 → a.CoolDown 폴백
_cd[a] = (_clock.Now + cd, cd);
```

- BaseCharacter.SlotCooldown(slot) = 슬롯 → Stat.GetData().QSkillCoolTime/ESkillCoolTime 매핑.
- 쿨 상태는 러너 인스턴스(캐스터별)가 소유 → 같은 E 에셋을 공유해도 캐릭터 간 쿨이 안 섞인다.

03 BaseCharacter — 입력=Commit, 비트=Fire

쿨·궁게이지는 입력 시점 소비(컷신 내내 쿨이 도는 게 보임), 이펙트는 애니 비트가 발사.

```
UseSkillQ → 궁 게이지 만 총 검사 → Commit(쿨 입력 시점 시작) → ConsumeUltimateGauge →
ChangeState(QSkillCutScene)
... 컷신(OnSkillEnter/OnCutsceneEnded 무변경) → Q_Skill 애니 ...
→ [AbilityBeat] → UseAbility(Q_Skill,beat) → Fire(이펙트)
```

- UseSkillE — Commit → ChangeState(E_Skill) → Fire (즉발). UseAbility 안 거침 → 쿨 시작 보장.
- UseAbility — Fire 전용(비트 경로). 쿨/코스트 안 건드림.
- 읽기 파사드 — SlotCooldownRemaining/Duration(slot), IsUltimateReady (에너지 만충 && Q 쿨 0).
- 평타 — 쿨 없이 **StateMachine**으로 게이팅(Attack 재진입 차단 = 애니 길이가 발사 간격). CharacterCombat 소멸.

04 UI — 러너 폴링으로 이관

쿨이 절대시각(무tick)이라 "쿨 끝남" 이벤트가 없어졌다 → 폴링.

UI	소스	방식
쿨 채움 링	러너 CooldownRemaining/Duration (파사드 경우)	매 프레임 폴링(GameSceneCanvas.Update)
궁 준비 글로우(활성)	bc.IsUltimateReady	UpdateCooldowns 폴링에 얹음
궁 준비 글로우(벤치)	bc.IsUltimateReady	PartySlotUI.Update() 폴링 → 절대시각 쿨 끝나는 순간 점등
에너지 바 · HP	OnEnergyChanged · OnHpChanged	이벤트 유지

미발동 스킬은 CooldownRemaining=0 → SkillIconUI 가 currentCool>0 일 때만 나눗셈 → 0 나눗셈 없음.

05 죽은 코드 정리

대상	처리	
CharacterCombat.cs	평타레이트·Q/E 포워딩 전부 흡수됨 → 파일 삭제	DELETE
CharacterStat	Q/E 쿨(Stat-Current-SO값세팅), TickCooldowns, TryUseSkillQ/E, CheckUltimateReadyState, _isUltimateReady, OnUltimateStateChanged 제거	삭제
PartyManager.TickCooldowns + Managers.Update 호출	절대시각이라 불필요 → 제거	삭제
BaseCharacter	_combat / _attackRate 필드·생성 제거. 러너 미사용 using 정리	삭제
ConsumeUltimateGauge · AddEnergy ·에너지 regen · OnEnergyChanged	유지	유지

06 검증 & 남은 것

에디터에서 필요

소스 레벨은 죽은 심볼 전수 grep 후 잔여 참조 0(단 `Stat.GetData().QSkillCoolTime` 은 `CharacterDataSO` 값이라 의도적 유지). **에디터 컴파일 + 플레이 실측 필요**. 특히 **Q 이펙트는 Q_Skill 애니 클립에 AbilityBeat 이벤트를 꽂아야** 나온다(쿨·궁게이지·글로우는 이벤트 없이 동작). 인스펙터 `_abilities` Q_Skill 슬롯 할당 확인.

남은 것 (선택)

E 발동을 AbilityBeat로 — 지금 E는 즉발. 투사체 E가 생기면 `SlotForState` 에 `E_Skill→E_Skill` + 비트 승격 (= Phase 2). · **CDR(쿨감)** — `_cd` 가 이미 `dur`를 저장하므로 `ctx.CoolDown = base × CDR` 만 곱하면 됨(Phase 4).